

G1 Physical-Robot Payload Campaign — Operator Guide

Twenty runs, four conditions, manual spill ground truth, cross-run belief state — everything needed to produce the env=real results for Section VII.H, mirroring the digital-twin campaign

Unitree G1 “Air” · real laboratory · July 2026 · companion to “G1 Digital-Twin Payload Experiment” (rev. 4)

1. What this campaign must produce

- **The env=real rows of Table 6:** time, spills/run (human-marked), collisions and completion per condition — directly comparable with the twin because metrics, dataset schema and analysis scripts are identical.
- **The Layer-2 recovery curve on real hardware** (the one result the twin could not produce): under the wrong-prior condition the fast analogy should genuinely cruise in the open lab (clearance ≈ 1.0 certifies it), gait-induced spills should occur, the persisted plausibility should erode run by run, and a later run should START pre-corrected. This is thesis Fig. 5.3.6.2-3 with physics.
- **Live $\hat{p}$ traces on hardware** (meta2_rho is logged automatically) for the certification-ratio figure and the future B-calibration.

Every run records automatically: per-tick samples (pose, clearance, progression, mobility, META2 action/analogy/tension/fulfilment/PI/ $\hat{p}$ /env), events (collisions, spills, governance transitions), telemetry, and camera frames. You only add: the spill marks, and one line of condition notes per run in runs_summary.csv afterwards.

2. Setup (once per session)

2.1 Hardware checklist

- ☐ Robot charged > **80%**, standing in walk mode, relocalized in the vendor app (SLAM screen, map loaded, camera ON).
- ☐ iPhone connected by USB to the Ubuntu machine, trusted; Web Inspector ON; do NOT open Safari's inspector on that page.
- ☐ Physical remote in hand at all times — **L2+B is the kill switch** (damping stop).
- ☐ 2-3 m of clear space around the route; **water only, never coffee**; floor dry.
- ☐ **The cup:** same cup for the whole campaign, radius ≈ 4 cm, filled to **1 cm below the rim** (matches the model freeboard). Refill to the same level before every run. Lid available for the sealed condition.

2.2 Software bring-up (Ubuntu, 3 terminals)

Terminal 0 — always first:

```
cd ~/G1_UNITREE_ROBOT_META_REASONING && git pull
ios_webkit_debug_proxy
```

Terminal 1 — perception server (required: the perception gate blocks navigation without verified vision):

```
source venv/bin/activate
G1_FLOORCOLOR=1 python perception_server.py --host 0.0.0.0 --port 8008 \
    --fx 300 --fy 300 --cx 160 --cy 120 --cam-h 1.10 --cam-pitch -10
```

Terminal 2 — the run (arm-specific command, §4). Confirm at start-up: `[perc] ... -> OK`, `PERC-TEST ... floorcolor=ON`, and the governance banner (`META2 ACTIVO ... analogia inicial ...`) when governed.

Terminal 3 — the spill marker (§3). Leave it running for the whole session.

3. Spill ground truth: the technician's marker

One person watches the cup for the entire run and marks **every spill event** — any amount of water leaving the cup counts as one event; a continuous slosh-over is one event, a second distinct slosh is a second event. When in doubt, mark it and write a note. The mark is timestamped into the run dataset (kind = spill_human) and counted into the summary (spills_human); it also feeds the governance fragility channel in real time, so **the robot itself reacts to marked spills** (one spill → caution; two → FALLBACK ceiling; three or more → HELP stop: that is correct behaviour, not a malfunction).

3.1 How to mark (physical robot — UDP channel)

The physical stack is ROS-free (it drives the robot through the vendor app's WebRTC session), so the marker is a UDP datagram to the listener that every run opens on port 7777. Three equivalent ways:

A. The marker tool (recommended) — press ENTER once per spill:

```
python spill_mark.py          # on the Ubuntu machine, or
python spill_mark.py <UBUNTU_IP> # from any laptop on the same LAN
```

B. One-liner (any machine with netcat):

```
echo spill | nc -u -w1 <UBUNTU_IP> 7777
```

C. ROS bridge (only if you are running the simulator, or a ROS 2 laptop with the forwarder) — in simulation this topic is forwarded automatically to the same UDP channel:

```
ros2 topic pub --once /spill_event std_msgs/msg/Empty
```

Verification before run 1 of the session: send one test mark while a throwaway run is active and confirm the Terminal-2 line `[spill-GT] DERRAME humano #1 ...` appears. Abort that throwaway run (Ctrl+C) — it saves a dataset; note it as a test in the CSV. If no line appears: check the IP, and that port 7777/UDP is not firewalled.

4. The campaign: 20 runs, four conditions

Alternate direction every run within an arm (A→B, then B→A — the robot is already at the far waypoint), and note the direction. Run the arms IN THIS ORDER (the wrong-prior arm must come after the operator is fluent with the procedure):

#	Arm (runs)	Cup	Purpose
R1	R-M0 baseline — no governance (4)	open	The ungoverned reference: fast, expected to spill in cruise. Comparable to twin M0.
R2	R-M2 payload governed (6)	open	The proposed system: full four-layer configuration, persisted state. Expected ~0 spills at a bounded time cost.
R3	R-WRONG prior + Layer 2 (6)	open	THE key arm: door-task (Efficient-preferred) configuration with an open cup — deliberately mis-calibrated. In the real lab the fast analogy certifies in open floor: expect early-run spills, plausibility erosion in the state file after each run, and a later run starting pre-corrected in Cautious.
R4	Sealed lid: 2 × M0 + 2 × M2-covered (4)	sealed	Lid-state delta on hardware: passive physics for M0, task-conditioned exploitation for the governed covered-delivery configuration.

Table 1. Campaign design (20 runs). If time allows, extend R2 and R3 first — they carry the paper.

4.1 Exact commands per arm (Terminal 2)

R1 — baseline (4 runs):

```
G1_META2=0 G1_PERC=127.0.0.1:8008 python g1_goto.py gotoviz B # then A, B, A
```

R2 — governed payload (6 runs). Delete the state file ONCE before run 1, never between runs:

```
rm -f meta2_state_real_m2.json # ONLY before run 1 of the arm
G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json \
G1_M2_STATE=meta2_state_real_m2.json G1_M2_L4=1 G1_M2_L3=1 \
G1_PERC=127.0.0.1:8008 python g1_goto.py gotoviz B # alternate B / A
```

R3 — wrong prior + Layer 2 (6 runs). Fresh state before run 1; keep it afterwards:

```
rm -f meta2_state_real_wrong.json # ONLY before run 1 of the arm
G1_META2=2 G1_M2_CFG=config_meta2_g1door.json \
G1_M2_STATE=meta2_state_real_wrong.json \
G1_PERC=127.0.0.1:8008 python g1_goto.py gotoviz B # alternate B / A
```

R4 — sealed lid (4 runs; put the lid on the cup):

```
G1_META2=0 G1_PERC=127.0.0.1:8008 python g1_goto.py gotoviz B # x2 M0
G1_META2=2 G1_M2_CFG=config_meta2_g1door.json \
G1_M2_STATE=meta2_state_real_lid.json \
G1_PERC=127.0.0.1:8008 python g1_goto.py gotoviz B # x2 covered
```

Notes: L3 in R3/R4 is optional (leave it off to keep the wrong-prior dynamics clean; R2 carries the L3 evidence). Do NOT set the simulation escalation windows — the real robot uses the validated defaults. All other safety layers (relocalisation guard, hard-guard, door engagement, escape) are ON by default; leave them.

5. Per-run checklist

Before

- ☐ Cup refilled to the mark (1 cm below rim); lid per condition; cup handed to the robot / mounted as in previous sessions.
- ☐ Robot at the start waypoint, relocalized (pose sane in the app), battery noted.
- ☐ Marker terminal alive; observer in position with line of sight to the cup for the whole route.
- ☐ Correct arm command with the right G1_M2_STATE file (never mix state files across arms).

During

- ☐ Mark every spill immediately (one press per event).
- ☐ If the robot stops and the log shows sustained HELP / meta2_experience_abort: that is a **governed abort** — a valid result. Do not restart the run; record it.
- ☐ Kill switch only for genuine safety (a fall risk, a person entering the space). Note any manual intervention.

After

- ☐ Terminal 2 prints the dataset path (`[dataset] travesia guardada -> ...`). Confirm `spills\_human` appears in the summary if you marked any.
- ☐ For R3: inspect the state file after EACH run and write down the plausibilities:

```
python -m json.tool meta2_state_real_wrong.json | grep -A3 m_match
```

- ☐ Every 2-3 runs: `git add dataset/ \*.json && git commit -m "real runs ..." && git push origin HEAD:main` (resolve non-fast-forward with `git pull --no-rebase` first).

6. What the theory predicts (check as you go)

Arm	Expected signature	If instead you see...
R-M0	Spills in open-floor cruise (gait slosh at 0.30 m/s), few/none at the door (slow there anyway); time $\approx$ 55-90 s per leg	No spills at all in 4 runs $\rightarrow$ cup fuller / marker definition — verify fill level and continue; the contrast arm is what matters
R-M2	$\approx$ 0 spills; visibly slower cruise (0.18 m/s ceiling, relaxed by L3 in open floor); FALLBACK lines near the door are normal	Repeated HELP aborts far from any obstacle $\rightarrow$ note run IDs; likely fragility over-marking or a QoE issue — finish the arm, we analyse offline
R-WRONG	Early runs: fast cruise + spills; state file PI(Efficient) dropping after each spill run (1.0 $\rightarrow$ $\sim$ 0.75 $\rightarrow$ $\sim$ 0.5 ...); once PI < 0.70 the ceiling blends down mid-arm; once PI < 0.45 a run STARTS in Cautious — write down WHICH run that is	No spills at all in runs 1-2 $\rightarrow$ the real cruise may be gentler than modelled: extend this arm before any other, and check the marker line appears on a deliberate test mark
R4 lid	M0: fewer spills, same speed. M2-covered: faster than R-M2 at $\approx$ 0 spills	—

The one plot we are after: PI(Efficient_Nav) per completed R3 run, from your noted values — the hardware version of Figure 5. If it emerges, the analogical-trust story is closed end-to-end.

7. Troubleshooting

Symptom	Cause / action
PERC-GATE BLOCKED at start	Perception server not up or camera off: check Terminal 1, app camera ON. (G1_NOVIS=1 bypasses — baseline arm only, never governed payload runs.)
No [spill-GT] line on a test mark	Wrong IP / UDP 7777 firewalled / marker aimed at localhost from another machine — use the Ubuntu IP.
RELOCALIZACION DUDOSA / robot will not start	Re-localize in the app at the waypoint; check the app map matches reality.

Robot stops mid-run, log shows META2 ... HELP	Governed abort — a RESULT. Record it, reset the robot to a waypoint, continue with the next run.
Repeated reloc jumps / STOP by guard	Known guard (G1_RELOCGUARD). Re-localize; if recurrent, note and continue — the dataset is still valid up to the stop.
git push rejected (non-fast-forward)	git pull --no-rebase, then push origin HEAD:main.
Wrong state file used for an arm	Note it immediately; that run moves to the other arm's count or is discarded — do not delete anything, the audit sorts it out.

8. After the campaign: hand the data back

- Final `git push` of all datasets + the three `meta2\_state\_real\_\*.json` files (they are data: they contain the trust trajectories).
- Fill `condition` and `notes` per run in runs_summary.csv (`python summarize\_runs.py` regenerates it; runs are tagged env=real automatically).
- Your handwritten PI values per R3 run (photo of the notebook is fine).

Analysis then mirrors the twin campaign automatically: Table 6 gains its env=real rows, Figure 5 gains the hardware trace, $\hat{p}$ gets its B-calibration data, and the sim $\leftrightarrow$ real closing paragraph of Section VII.H writes itself from the comparison.

Companion documents: “G1 Digital-Twin Payload Experiment” rev. 4 (methods & theory), HANDOFF.md §8.18 (flags reference), COMO_LANZAR.md (simulator). All versioned with the stack.